FDS - M1 Informatique - HAI712I - Ingénierie Logicielle

Année 2025-2026, session 1 de janvier 2026.

Durée : 2h00. Documents non autorisés. Toutes les parties sont indépendantes, ordonnez les comme il vous convient. Le barême est globalement indicatif. La précision et concision des réponses est prise en compte ; choisissez bien les termes utilisés et quand 1 terme suffit, ne pas en donner 10 en espérant que l'un d'eux soit la bon.

Contexte. Plaçons nous dans le contexte de la réalisation de *Pict*, un framework jouet pour faire des images ou des dessin. Un dessin y est un ensemble de formes ; les formes sont à la base des lignes, des cercles ou des rectangles. Toute instance de la classe `Dessin` possède un titre et une collection de formes. Une forme est représentée par une instance d'une sous-classe de la classe `Forme`. *Pict* est à la fois un logiciel utilisable (il est opérationnel) et à la fois un framework utilisable par des programmeurs qui peuvent l'étendre pour lui ajouter ou pour adapter ses fonctionnalités.

Nous discutons dans cet exercice d'une implantation en *Java* de *Pict* et les questions portent sur sa réalisation, son architecture et ses qualités logicielles. Pour simplifier le propos, on n'écrit pas de vraies méthodes de dessin ; on décrit un dessin par un texte. La méthode (`drawDescription()`) affiche au terminal une description textuelle (une chaîne de caractères) d'un dessin. L'implantation des différente sortes de formes (Ligne, Cercle, etc) utilise la classe prédéfinie `java.awt.Point`. Un tel point possède une abscisse et d'une ordonnée). Une ligne (un segment de droite) peut ainsi être représentée par deux points (une origine et une extrémité), un cercle par un point (son centre) et un entier (son rayon), un rectancle par deux points (son point supérieur gauche et son point inférieur droit) ; etc.

Le listing 2 donne des extraits du code des classes `Dessin` et `Forme` avec leur méthode respective `drawDescription()`. Ces 2 classes dans cette version représentent l'architecture de base du framework *Pict*. Le listing 1 donne un exemple d'utilisation de ces classes [a].

a. Précision : on suppose que la redéfinition de la méthode `toString()` sur la classe `java.awt.Point` rend la chaine : "abscisse@ordonnée".

```
1  Dessin d1 = new Dessin("ballonBaudruche");
2  d1.add(new Cercle(new Point(5,5),1));
3  d1.add(new Ligne(new Point(5,2), new Point(5,5)));
4  d1.drawDescription(); // affiche le texte ci-dessous
   :
5  ...
6  Un dessin de ballonBaudruche, obtenu par :
7  - un cercle de centre: 5.0@5.0 et rayon 1.0
8  - une ligne de 5.0@2.0 à 5.0@5.0
```

Listing 1 – exemple de création d'un dessin.

```
1  class Dessin {
2      protected List<Forme> contenu; protected String title; //deux attributs
3      public Dessin(String t){ contenu = new ArrayList<Forme>(); title = t;} //un constructeur
4      public void add(Forme s) { contenu.add(s); }
5      public void remove(Forme s){ contenu.remove(s); }
6      public void drawDescription(){
7          System.out.println(this.getDescription() + " obtenu par : ");
8          for (Forme sh : contenu) sh.drawDescription(); }
9      public String getDescription(){
10         return ( "Un dessin de " + title ); }
11     ...}

13 public abstract class Forme{
14     public void drawDescription() { System.out.println (this.getDescription());}
15     public abstract String getDescription();
16     public abstract double surface();
17     ... }
```

Listing 2 – code partiel des classes `Dessin` et `Forme`

A Réutilisation - Environ 6 points

Considérons les listings 1, 2 et 3.

1. Dans le listing 3, on trouve deux affectations polymorphiques.

   a) Dites où elles sont.

   b) Cette sorte d'instruction est-elle fréquemment utilisés dans les frameworks ? si oui, pourquoi (qu'a-t-elle d'important pour un framework) ?

```
1 Forme f;
2 f = new Cercle(new Point(5,5),1));
3 f.drawDescription();
4 f = new Ligne(new Point(5,2), new
      Point(5,5));
5 f.drawDescription();
```

Listing 3 –

2. Dans le listing 3, on trouve la même instruction d'envoi de message `f.drawDescription()` 2 fois.

   a) L'exécution de chacune de ces deux instructions invoque une méthode de nom `drawDescription()`. En considérant le code des listings 2 et 3, et en se mettant à la place du compilateur réalisant une analyse statique du code, y-a-t-il une ambiguité quand à savoir quelle méthode `drawDescription()` va être appelée ? Autrement dit, le compilateur doit-il compiler ces deux instructions en activant la liaison dynamique, ou bien est-ce inutile ?

   b) On dirait que la méthode exécutée est paramétrée par le texte à afficher. Pourtant la méthode `drawDescription()` ne possède aucun paramètre apparent. Expliquez ce phénomène.

3. Imaginez le code de la classe `Cercle`, sous-classe de Forme, permettant d'obtenir le résultat d'exécution montré au listing 1.

   a) Est-il nécessaire de redéfinir la méthode `drawDescription()` sur Cercle ? si non pourquoi ? Si oui, écrivez le code de cette redéfinition en me disant pourquoi vous êtes sûr que c'en est bien une (redéfinition).

   b) Même question exactement pour `getDescription()`.

4. Décrivez comment vous organiseriez un framework de jeu de tests **Junit** pour un **framework** comme *Pict* (notez qu'il est question de réutilisation). Donnez le code d'une méthode de test applicable aux formes et une des méthodes `setUp()`.

B Réutilisation et typage statique - Environ 5 points

Pour optimiser l'impression des dessins, on souhaite que la collection des formes (attribut `contenu` de *Dessin* de type `ArrayList`) constituant un dessin soit triée selon la surface des formes, par ordre décroissant.

Pour cela, chaque sous-classe de `Forme` définit une méthode `surface()` [1].

Pour trier, on va utiliser la méthode statique `sort` de la classe `java.util.Collections` (avec un "s" à la fin, à ne pas confondre avec `java.util.Collection`) qui définit des méthodes statiques qui opèrent sur les collections.

Pour rappel, voici la signature et la documentation officielle de la méthode `sort(..)`.
- `public static <T extends Comparable<? super T>> void sort (List<T> list)`
- "*Sorts the specified list into ascending order, according to the natural ordering of its elements. All elements in the list must implement the Comparable interface. Furthermore, all elements in the list must be mutually comparable (that is, e1.compareTo(e2) must not throw a ClassCastException for any elements e1 and e2 in the list).*"

On note que cette méthode `sort(..)` fonctionne si les éléments de la collection `list` à trier sont de type `Comparable<...>`. Vous connaissez le type `Comparable` que nous avons utilisé pour faire les dictionnaires triés dans la séance 4 de TP. Ce type définit la méthode `compareTo(...)`. Si e1 et e2 sont des `Comparables<...>`, `e1.compareTo(e2)` rend -1 ou 0 ou 1 selon que e1 est inférieur, égal ou supérieur à e2.

1. En utilisant les versions des classes de base données au listing 2, on essaye de définir sur la classe `Dessin` la méthode `sort` suivante :

1. Pour une ligne, on supposera que la surface est la distance entre le point origine et le point extrémité. Note : il ne vous est pas demandé d'écrire les méthodes `surface()` ; on suppose qu'elles existent.

```
public void sort() { Collections.sort(contenu); }
```

Listing 4 –

Le compilateur refuse en affichant le message d'erreur suivant : *"Bound mismatch : The generic method* `sort(List<T>)` *of type* `Collections` *is not applicable for the arguments (*`ArrayList<Forme>`*). The inferred type Forme is not a valid substitute for the bounded parameter* `<T extends Comparable<? super T>>`*".*

- a) Expliquez l'erreur.

- b) Modifiez ce qui doit l'être pour que cette définition de methode dans la classe `Dessin` compile sans erreur.

2. Modifiez ce qui doit l'être partout où c'est nécessaire, pour que les instructions du listing 5 s'exécutent comme indiqué au listing 6.

```
Dessin d2 = new Dessin("TestDeTri");
d2.add(new Cercle(new Point(5,5),2));
d2.add(new Ligne(new Point(5,2), new
    Point(5,9)));
d2.add(new Cercle(new Point(7,7),1));
d2.add(new Ligne(new Point(5,2), new
    Point(5,5)));
d2.drawDescription();
d2.sort();
System.out.println("--- après tri");
d2.drawDescription();
```

Listing 5 –

```
Un dessin de TestDeTri obtenu par :
– un cercle de centre: 5.0@5.0 et rayon 2.0
– une ligne de 5.0@2.0 à 5.0@9.0
– un cercle de centre: 7.0@7.0 et rayon 1.0
– une ligne de 5.0@2.0 à 5.0@5.0
--- après tri
Un dessin de TestDeTri obtenu par :
– une ligne de 5.0@2.0 à 5.0@5.0
– un cercle de centre: 7.0@7.0 et rayon 1.0
– une ligne de 5.0@2.0 à 5.0@9.0
– un cercle de centre: 5.0@5.0 et rayon 2.0
```

Listing 6 –

3. Expliquez la contrainte suivante sur le type T : `<T extends Comparable<? super T>>`. Expliquez en quoi vous avez respecté cette contrainte dans votre réalisation.

C Une modification d'architecture pour des dessins arborescents - Environ 4 points

1. Modifiez l'architecture de base de *Pict*, pour permettre qu'un dessin puisse être composé d'un dessin et ce sans que la méthode `drawDescription()` de la classe `Forme` n'ait à être modifiée. Ceci permettra d'exécuter les instructions du listing 7 pour donner le résultat du listing 8.

   Décrivez un solution précise et donnez les éléments clé du code (pas les détails) faisant que cela fonctionne.

```
Dessin d3 = new
    Dessin("croquis_annexe");
d3.add(new Cercle(new Point(5,5),1));
d3.add(new Cercle(new Point(7,7),1));
Dessin d4 = new
    Dessin("croquis_principal");
d4.add(new Cercle(new Point(6,6),1));
d4.add(d3);
d4.drawDescription();
```

Listing 7 –

```
Un dessin de croquis_principal obtenu par :
– un cercle de centre: 6.0@6.0 et rayon 1.0
– Un dessin de croquis_annexe obtenu par :
  – un cercle de centre: 5.0@5.0 et rayon 1.0
  – un cercle de centre: 7.0@7.0 et rayon 1.0
```

Listing 8 –

2. Améliorez, si-besoin, l'architecture que vous avez proposée (ou expliquez en quoi elle suffit) afin de pouvoir définir une nouvelle méthode `dessinsEnglobants()` rendant, pour une forme donnée, la collection des dessins auquel elle appartient. Pour le "cercle de centre : 7.0@7.0" du listing 7, le résultat serait une collection contenant `d3` et `d4`.

   Vous avez liberté de choix pour décider si l'on peut, ou pas, mettre un dessin dans plusieurs autres, ou une forme dans plusieurs dessins distincts. Tout commentaire lié à un(des) schéma(s) de conception en

rapport avec la question sera intéressant.

D Ajout de fonctionnalités à un objet individuel : Environ 5 points

On souhaite pouvoir ajouter puis éventuellement retirer, à l'exécution, des fonctionnalités à tout objet de type `Forme`, sans modifier aucune des classes existantes de l'architecture de base donnée en préambule à ce sujet, ni leur méthodes, car ces ajouts d'informations supplémentaires ne les concerne pas toutes.

Par exemple, on souhaite que la description d'une forme, choisie individuellement, affiche des informations supplémentaires, comme un commentaire de ce qu'elle représente dans un dessin, ou sa couleur. On voit dans le listing suivant une application de cette idée où on ajoute à un cercle le commentaire qu'il représente un ballon et qu'il est de couleur rouge, sans avoir modifié la classe `Cercle`.

```
1  Dessin b = new Dessin("un ballon de baudruche");
2  b.add(new Commentaire("le ballon",
3          new Couleur(Color.red,
4                  new Cercle(new Point(3,4), 1))));
5  b.add(new Commentaire("la ficelle",
6          new Ligne(new Point(3,0), new Point(3,3))));
7  b.drawDescription() // produit le texte suivant :
8  "Un dessin de un ballon de baudruche obtenu par :
9  - un cercle de centre: 3.0@4.0 et rayon 1.0, de couleur: rouge (le ballon)
10 - une ligne de 3.0@0.0 à 3.0@3.0 (la ficelle)"
```

Listing 9 – Création de formes avec des fonctionnalités additionnelles.

1. Proposez une architecture pour réaliser cette spécification.
2. Dans cette architecture, donnez le code de la classe `Commentaire`.
3. Faites un dessin représentant en mémoire l'objet référencé par la variable `b` du listing 9 après que l'instruction aux lignes de code 5 et 6 a été exécutée ; vous montrerez aussi les objets référencés dans les attributs de celui-ci.
4. En quoi l'associativité de l'opérateur Java de concaténation des `String` est-elle importante dans cet exercice ?
5. Connaissez vous d'autres programmes où le schema que vous avez utilisé l'est aussi ?